

ABC455 D - Card Pile Query 题解

题意概括

有 N 张卡片和 N 堆牌。初始时第 i 堆里只有卡片 i 。

每次操作给出 C 和 P ：

- 把卡片 C 以及它上面的整段牌一起搬走；
- 按原顺序叠到卡片 P 的上方。

题目保证操作前 C 和 P 不在同一堆，并且 P 一定是某一堆的顶部。求所有操作结束后每堆的牌数。

解题思路

如果每次都真的维护“每堆有哪些卡”，一次操作可能会改很多元素，无法通过。

注意题目只要求最后每堆的大小，不要求中途随时查询。于是可以只维护每张牌上下相邻的是谁。

为了让“每堆底部”也统一成一张牌，我们给每堆加一张不会移动的虚拟底牌：

- 实际卡片编号： 0 到 $N-1$
- 第 i 堆的虚拟底牌编号： $N + i$

然后维护两个数组：

- $up[x]$ ：卡片 x 正上方的卡片编号，没有则为 -1
- $down[x]$ ：卡片 x 正下方的卡片编号，没有则为 -1

初始时：

- $up[N + i] = i$
- $down[i] = N + i$

现在处理一次操作，把 C 那一整段挪到 P 上方。设 $D = down[C]$ ，也就是原来 C 正下方的牌。

更新三处就够了：

1. $down[C] = P$
2. $up[P] = C$
3. $up[D] = -1$

含义分别是：

- C 这一段的底部现在压在 P 上；

- P 上面现在接着 C；
- 原来留在下面的那一堆，顶部变成了 D。

最后，从每个虚拟底牌一路沿着 up 往上走，数一数经过了多少张实际卡片，就是该堆最终大小。

正确性说明

我们维护的是每张牌与上下相邻牌的关系。

对一次操作而言，被移动的是“从 C 到当前堆顶”的整段：

- 这段内部的相对顺序完全不变，所以段内其他相邻关系不需要修改；
- 只有这段底部与原堆、目标堆的连接关系发生变化。

因此只需要改动：

- C 与新下方 P 的连接；
- P 与新上方 C 的连接；
- 原下方 D 的上方变为空。

更新完成后，所有堆的链表结构都与真实状态一致。

最终从每个虚拟底牌向上遍历，恰好能走到该堆中的所有实际卡片，而且每张卡片只属于唯一一堆，所以得到的计数就是最终答案。算法正确。

复杂度

每次操作只修改常数个位置，最后每张牌也只会遍历一次。

- 时间复杂度： $O(N + Q)$
- 空间复杂度： $O(N)$

参考实现

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int n, q;
    cin >> n >> q;

    vector<int> up(2 * n, -1), down(2 * n, -1);
```

```
for(int i=0; i<n; i++){
    up[n + i] = i;
    down[i] = n + i;
}

while(q--){
    int c, p;
    cin >> c >> p;
    c--;
    p--;

    int d = down[c];
    down[c] = p;
    up[p] = c;
    up[d] = -1;
}

for(int i=0; i<n; i++){
    int x = n + i;
    int ans = 0;

    while(up[x] != -1){
        x = up[x];
        ans++;
    }

    cout << ans << " \n"[i == n - 1];
}

return 0;
}
```